

Practica 4

Gramática para Analizador Sintáctico

Sarah Sophia Olivares García

1. Conjuntos N , Σ , y símbolo inicial S

Dada la gramática:

$$\begin{aligned}P = \{ & S \rightarrow \text{Expr} \mid \text{Asig}, \\ & \text{Expr} \rightarrow \text{Term Expr}', \\ & \text{Expr}' \rightarrow +\text{Term Expr}' \mid -\text{Term Expr}' \mid \varepsilon, \\ & \text{Term} \rightarrow \text{Factor Term}', \\ & \text{Term}' \rightarrow *\text{Factor Term}' \mid /\text{Factor Term}' \mid \varepsilon, \\ & \text{Factor} \rightarrow \text{Num} \mid \text{Var} \mid (\text{Expr}) \mid -\text{Expr}, \\ & \text{Num} \rightarrow \text{Entero Decimal}, \\ & \text{Decimal} \rightarrow \text{.Entero} \mid \varepsilon, \\ & \text{Entero} \rightarrow \text{Digito} \mid \text{Digito Entero}, \\ & \text{Digito} \rightarrow 0 \mid 1 \mid \dots \mid 9, \\ & \text{Asig} \rightarrow \text{var Var} = \text{Expr}, \\ & \text{Var} \rightarrow \text{Letra Pos}, \\ & \text{Pos} \rightarrow \text{Var} \mid \varepsilon, \\ & \text{Letra} \rightarrow _ \mid a \mid b \mid \dots \mid z \mid A \mid B \mid \dots \mid Z\}\end{aligned}$$

- N : $\{S, \text{Expr}, \text{Expr}', \text{Term}, \text{Term}', \text{Factor}, \text{Num}, \text{Decimal}, \text{Entero}, \text{Digito}, \text{Asig}, \text{Var}, \text{Pos}, \text{Letra}\}$
- Σ : $\{+, -, *, /, (,), ., \text{var}, =, _, \text{a-z}, \text{A-Z}, 0-9\}$
- Símbolo inicial: S

2. Eliminación de ambigüedad

La gramática original presenta ambigüedad en la expresión “ $\text{Expr} \rightarrow \text{Term Expr}'$ ”, ya que podría generar conflictos en la interpretación de operadores. Para eliminar la ambigüedad, utilizamos una jerarquía de operadores con precedencia y asociatividad definida:

- Operadores de suma y resta tienen menor precedencia que los de multiplicación y división.
- La exponenciación es asociativa por la derecha.

Esto se puede manejar mediante reglas de precedencia en BYACC/J.

3. Eliminación de recursividad izquierda

La regla $\text{Expr} \rightarrow \text{Term Expr}'$ no presenta recursividad izquierda directa o indirecta. Por tanto, no es necesario realizar modificaciones.

4. Factorización izquierda

No se observa factor común entre las producciones. Por lo tanto, no se requiere factorización izquierda.

5. Nuevos conjuntos N y P

Los conjuntos N y P permanecen iguales debido a que no hubo necesidad de eliminar recursividad o realizar factorizaciones.

6. Manejo de conflictos Shift/Reduce

Los conflictos de shift/reduce se resolverán utilizando las declaraciones de precedencia y asociatividad en BYACC/J.

7. Definición en BYACC/J

El archivo resultante en BYACC/J es el siguiente:

```
%{
    import java.lang.Math;
    import java.io.*;
%}

%token NUM VAR
%left '+' '-'
%left '*' '/'
%right '^'

%%

input: /* empty */
      | input line
;

line: '\n'
     | expr '\n' { System.out.println("Resultado: " + $1); }
;

expr: term
     | expr '+' term { $$ = $1 + $3; }
     | expr '-' term { $$ = $1 - $3; }
;

term: factor
     | term '*' factor { $$ = $1 * $3; }
     | term '/' factor { $$ = $1 / $3; }
;

factor: NUM
       | VAR
       | '(' expr ')' { $$ = $2; }
       | '-' factor %prec '-' { $$ = -$2; }
```

;

%%

8. Ejemplo de entrada y salida

Archivo de prueba (prueba.txt):

3 + 5

(2 * 6) - 4

Salida esperada:

Resultado: 8

Resultado: 8